

企业级应用软件设计与开发

课程大作业报告

SmartFlow

AI 驱动的企业审批智能体系统

课程名称	企业级应用软件设计与开发
项目名称	SmartFlow: AI 驱动的企业审批智能体系统
方向	方向一: Agentic AI 原生开发
学号	2025302978
姓名	丰宇硕
专业	计算机技术
指导教师	威欣
提交日期	2026 年 6 月 22 日

代码仓库: <https://github.com/ffyyssw/smartflow>

摘要

传统企业审批系统通常依赖用户先理解流程分类、表单字段和审批规则，再逐项填写。该模式在请假、报销、采购等高频办公场景中容易出现流程入口选择错误、字段遗漏、材料不齐、审批链配置不透明等问题。SmartFlow 将自然语言理解能力嵌入企业审批软件，通过 Agent 工作流完成“流程识别—字段抽取—规则检查—审批路由规划—摘要生成—结构化响应”，并与表单、附件、审批步骤、审计日志和角色权限形成闭环。

项目采用 Vue 3、Vite、Element Plus 构建前端，使用 FastAPI、SQLAlchemy 2.0、Pydantic V2 和 JWT 构建后端，以 LangGraph 编排 Agent 节点，并支持 OpenAI 兼容接口和本地 Mock 规则模式双路径。系统覆盖请假、报销、采购三类流程，支持员工发起、经理/HR/财务/采购等角色审批、申请详情与时间线展示。本文按照 Product Spec、Architecture Spec、API Spec、系统架构、关键实现、测试评估、升级路线和课程总结组织，对代码中的主要设计决策、边界规则和潜在一致性问题进行分析。

关键词：Agentic AI；企业审批；LangGraph；FastAPI；Vue 3；工作流；自然语言表单

目 录

一、选题背景与设计思想	4
1.1 问题定义	4
1.2 传统方案不足与现状分析	4
1.3 项目价值	5
1.4 设计目标与原则	5
1.5 技术路线	5
1.6 项目范围与边界	5
二、Specs 规范文档（SDD 核心）	6
2.1 Product Spec（产品规格）	6
2.2 Architecture Spec（架构规格）	8
2.3 API Spec（接口规格）	9
2.4 数据规格与状态规格	11
2.5 需求—设计—实现追踪矩阵	12
三、系统架构与设计	13
3.1 核心架构图	13
3.2 Agent 交互流程	14
3.3 业务审批交互流程	14
3.4 数据流设计	15
3.5 部署与运行拓扑	16
四、关键实现与代码展示	16
4.1-4.5 Agent 与 LLM 关键实现	16
4.6-4.8 规则、状态推进与配置	18
4.9 AI IDE 使用过程证据	20
五、测试与评估	21
5.1-5.3 测试用例与 Agent Benchmark	21
5.4-5.7 边界、安全、缺陷与 Demo	23
六、系统升级与扩展	25
七、课程总结	27
附录 A：部署与演示操作手册	29
附录 B：提交前检查清单	30
附录 C：参考资料与源码定位	31

一、选题背景与设计思想

1.1 问题定义

SmartFlow 面向企业内部请假、报销和采购等结构化审批场景，核心问题不是简单地“做一套表单”，而是降低员工理解制度和选择流程的认知成本，并保证审批过程可追踪、可解释、可审计。用户只需描述业务意图，例如“下周一请病假三天，工作交接给小王”，系统自动识别流程类型、抽取日期与原因、判断证明材料、规划审批角色链，并生成可编辑的动态表单。

该问题可形式化为：给定自然语言输入 x 、企业流程模板 P 和规则集合 R ，Agent 输出结构化结果 $y=\{process_type, confidence, extracted_fields, missing_fields, required_materials, approval_route, risk_tips, title, summary\}$ ；用户确认后，业务系统将 y 映射为申请实体、附件实体和有序审批步骤，并根据审批动作驱动状态迁移。

源码依据：README 项目说明；backend/app/ai/state.py；backend/app/schemas/ai.py。

1.2 传统方案不足与现状分析

不足	传统表现	SmartFlow 对应设计
流程入口依赖人工判断	员工需先判断“请假/报销/采购”，选错后重新填写	意图分类节点自动识别流程类型并给出置信度
字段多且语义分散	表单字段由用户逐项回忆，容易遗漏日期、金额、事由等	实体抽取节点将自然语言映射到流程字段，并返回 missing_fields
制度规则不可见	材料要求、金额阈值、审批链常散落在制度文档中	规则检查与路由规划将材料、风险、审批角色链前置展示
审批状态不透明	用户只能看到“处理中”，难以定位当前审批人和下一步	ApprovalStep 有序建模，详情页展示审批时间线
LLM 依赖与演示风险	无 API Key 或网络异常时智能功能不可用	真实 LLM 与 Mock 规则模式双路径，异常时自动回退
系统审计薄弱	审批意见与附件操作缺少统一留痕	AuditLog、附件元数据和操作 IP 形成审计链

1.3 项目价值

- 业务价值：将制度知识从“用户记忆”转化为“系统提示”，减少选错流程、漏填字段和材料补交。
- 管理价值：审批链由规则计算并固化为步骤，提升权限边界、流程透明度与审计可追溯性。
- 技术价值：展示 Agent 编排、结构化输出、规则引擎、业务状态机、前后端分离与数据库持久化的完整结合。
- 教学价值：项目同时覆盖需求规格、架构设计、API、数据建模、AI 工程化、测试与演进路线，符合企业级软件大作业的完整性要求。

1.4 设计目标与原则

目标/原则	具体要求	可验证指标
自然交互	支持 1-2000 字自然语言输入，自动识别三类流程	返回 process_type、confidence 与结构化字段

目标/原则	具体要求	可验证指标
确定性约束	金额、天数、材料、审批链由规则函数计算	同一规范化输入应得到一致路由
可降级	无 Key、LLM 请求失败或 JSON 异常时回退 Mock	核心演示不依赖外部服务
前后端解耦	统一 API 与响应格式，前端通过 api 模块访问	接口可由 Swagger 独立验证
安全与审计	JWT 鉴权、角色校验、上传限制、操作日志	未认证请求拒绝；越权审批阻断；关键动作留痕
可扩展	流程模板、规则、节点、数据库和模型接口可替换	新增流程尽量不修改既有主链路

1.5 技术路线

- 1. 以企业审批业务为边界，定义请假、报销、采购三类流程和角色体系。
- 2. 使用 LangGraph 将 AI 分析拆分为意图分类、实体抽取、规则检查、路由规划、摘要生成与响应格式化。
- 3. 通过 Pydantic 约束 API 输入，通过 SQLAlchemy 模型持久化用户、申请、审批步骤、附件和审计日志。
- 4. 前端根据 process_type 与 form_schema 渲染动态表单，并通过 Axios 拦截器携带 JWT。
- 5. 以审批步骤状态机驱动通过、驳回、退回和下一节点激活，构成完整业务闭环。
- 6. 以 Mock 规则引擎保证离线可演示，以 OpenAI 兼容接口提升自然语言泛化。

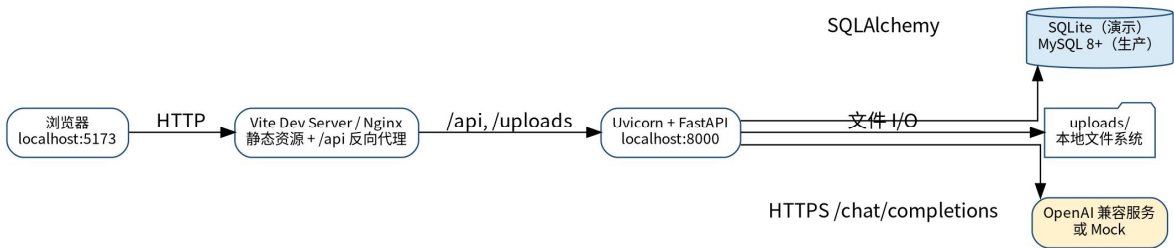


图 1-1 SmartFlow 开发/部署技术路线

1.6 项目范围与边界

范围内	范围外/当前限制
请假、报销、采购三类标准审批	复杂 BPMN 自由编排与跨组织多租户
员工、经理、HR、财务、采购、管理员等角色	企业统一身份认证、LDAP/SSO、组织架构同步
自然语言分析、动态表单、材料提示、审批链推荐	对发票/病假证明的 OCR 与真实性核验
SQLite 演示与 MySQL 切换	高并发分布式事务、消息队列、对象存储与容灾
本地上传、审批时间线和审计日志	电子签章、法务合规归档与长期保全

二、Specs 规范文档（SDD 核心）

2.1 Product Spec（产品规格）

2.1.1 产品定位与目标用户

产品定位为“AI 驱动的轻量级企业审批与办公流程助手”。其关键差异不是新增一个聊天机器人，而是将自然语言理解结果直接映射到可提交、可审批、可审计的企业业务对象。

角色	主要目标	核心权限/场景
员工 employee	快速发起合规申请并跟踪进度	AI 分析、补全表单、上传附件、查看本人申请
经理 manager	处理部门内首级审批	查看待审批、通过/驳回/退回、填写审批意见
HR hr	审核请假制度与证明材料	处理请假流程中的 HR 节点
财务 finance	核验报销与高额采购财务风险	处理报销或采购财务节点
采购 procurement	处理采购合规与供应商事项	处理采购流程节点
管理员 admin	兜底审批、系统演示和规则管理	查看全局、处理无匹配审批人场景

2.1.2 用户故事与验收条件

编号	用户故事	验收条件
US-01	作为员工，我希望用一句话描述需求，以便自动选择流程。	输入非空文本后返回受支持的流程类型与置信度。
US-02	作为员工，我希望系统自动填充已提及字段。	日期、金额、数量、原因等能够进入 <code>extracted_fields</code> 。
US-03	作为员工，我希望提交前知道缺什么。	返回 <code>missing_fields</code> 、 <code>required_materials</code> 和 <code>risk_tips</code> 。
US-04	作为员工，我希望知道申请将经过哪些角色。	返回有序 <code>approval_route</code> ，并在详情中生成对应步骤。
US-05	作为审批人，我希望只处理自己有权处理的当前步骤。	非当前审批人或非 <code>pending</code> 步骤的动作被拒绝。
US-06	作为申请人，我希望看到完整审批轨迹。	详情包含步骤状态、审批人、意见、时间和审计记录。
US-07	作为演示者，我希望没有 LLM Key 时仍能运行。	<code>LLM_API_KEY</code> 为空时自动使用 Mock，AI 分析接口可返回完整结构。

2.1.3 功能需求

编号	功能	规格描述	优先级
FR-01	认证登录	用户名/密码登录，返回 <code>bearer token</code> 与用户信息	高
FR-02	当前用户	验证 <code>token</code> 并返回当前用户	高
FR-03	AI 分析	自然语言→流程、字段、缺失项、材料、路由、风险、摘要	高
FR-04	流程模板	查询流程类型、表单 <code>schema</code> 与规则	高
FR-05	动态表单	根据流程 <code>schema</code> 渲染与校验表单	高
FR-06	附件上传	限制扩展名/大小，保存元数据并关联申请	高
FR-07	创建申请	生成申请编号，保存 AI 分析，初始化审批步骤	高
FR-08	我的申请	分页并按状态/类型筛选	中

编号	功能	规格描述	优先级
FR-09	申请详情	展示表单、附件、步骤、时间线与日志	高
FR-10	待审批	按用户角色/审批人筛选当前 pending 步骤	高
FR-11	审批操作	支持 approve/reject/return，意见必填	高
FR-12	状态推进	通过后激活下一步；末步完成；驳回/退回终止或回转	高
FR-13	仪表盘	提供申请和待办统计	中
FR-14	规则可解释	展示材料依据、风险提示和审批角色链	中

2.1.4 非功能需求

维度	规格	当前实现/建议
安全性	所有业务接口除登录外需认证；敏感配置不得提交仓库	JWT HS256；生产环境应更换 SECRET_KEY 并启用 HTTPS
可靠性	外部 LLM 不可用时核心流程仍可工作	节点捕获异常并回退 Mock；建议增加日志与熔断指标
性能	常规 API P95 < 500 ms；LLM 分析 P95 < 8 s	本地规则应毫秒级；LLM 客户端超时 30 s，建议改为异步与分级超时
可维护性	路由、服务、模型、Schema、AI 节点分层	现有目录分层清晰；应增加测试目录和规则版本号
兼容性	开发环境支持 Windows，数据库可切 SQLite/MySQL	README 提供 PowerShell 启动命令与 DATABASE_URL 切换
可观测性	记录请求、LLM 回退、审批状态变化和异常	已有审计模型；仍需结构化日志、trace_id、指标面板
可用性	表单错误提示清晰，401 自动回登录	前端拦截器清理 token；建议增加草稿自动保存

2.2 Architecture Spec（架构规格）

2.2.1 分层与组件职责

层次	组件	职责	依赖方向
表示层	Vue Views / Components / Router / Pinia	页面、路由守卫、状态管理、动态表单、时间线	仅调用前端 api 模块
接口层	FastAPI Routers / Schemas / Deps	HTTP 路由、参数校验、JWT 解析、统一响应	调用服务或 Agent
Agent 层	workflow / nodes / prompts / state / LLMClient / Mock	自然语言分析与结构化决策	调用 LLM 与规则函数
领域服务层	application / approval / auth / upload service	事务、权限、编号、状态推进、序列化	访问模型与工具函数
持久化层	SQLAlchemy Models / Session / SQLite/MySQL	业务数据持久化与关系维护	不依赖上层
外部能力层	OpenAI-compatible endpoint / file system	模型推理、附件存储	通过适配器调用

2.2.2 关键架构决策（ADR 摘要）

ADR	决策	理由	代价/风险
-----	----	----	-------

ADR	决策	理由	代价/风险
ADR-01	Agent 与业务提交分离	AI 仅预填，用户确认后才创建申请	需维护分析结果与最终表单差异
ADR-02	结构化 JSON 作为 Agent 边界	便于映射与测试	LLM JSON 不稳定，需解析、校验和回退
ADR-03	LangGraph 顺序节点编排	节点职责清晰，便于调试与后续条件分支	当前图为线性链，自治性和工具调用循环有限
ADR-04	LLM + Mock 双模式	无 Key 可演示，外部故障可降级	两套逻辑可能产生结果差异，需要一致性测试
ADR-05	JSON 保存 form_data/ai_analysis	三类流程字段不同，快速迭代	复杂查询和字段级约束较弱
ADR-06	审批步骤实例化	提交时固化审批链，便于追踪和审计	规则变更不会自动作用于历史实例
ADR-07	SQLite 默认、MySQL 可切换	降低本地运行门槛，保留生产迁移路径	SQLite 并发写入能力有限

2.2.3 安全、异常与降级规格

- 认证：登录成功生成 JWT，载荷包含 sub、role、user_id；后续接口通过 Depends(get_current_user) 获取用户。
- 输入校验：AI 文本 1-2000 字；申请标题 1-200 字；审批意见 1-500 字；process_type 仅允许 leave、reimbursement、purchase。
- 授权：申请详情和审批动作应由服务层检查“本人/有权角色/当前步骤/步骤状态”，不能仅依赖前端隐藏按钮。
- 文件安全：限制允许扩展名、最大体积与存储路径；生产环境需增加 MIME 嗅探、病毒扫描和对象存储签名 URL。
- LLM 降级：API Key 为空或调用/解析异常时转为 Mock；建议为每次回退记录 reason、model、latency 与 trace_id。
- 数据一致性：创建申请与审批步骤、审批动作与下一步激活应放在同一数据库事务中。

2.3 API Spec（接口规格）

2.3.1 通用约定

项目	规范
Base URL	开发环境 http://localhost:8000；业务前缀 /api
认证	Authorization: Bearer <access_token>
内容类型	JSON 接口 application/json；上传 multipart/form-data
统一响应	{ "code": 200, "message": "操作成功", "data": ... }
错误处理	HTTP 状态码与 code 同步；校验错误由 FastAPI/Pydantic 返回；业务错误使用明确中文 message
分页	page >= 1；1 <= page_size <= 50
命名	流程码使用 leave / reimbursement / purchase；状态使用 pending / waiting / approved / rejected / returned 等

2.3.2 接口总表

方法	路径	认证	说明
POST	/api/auth/login	否	登录并返回 JWT
GET	/api/auth/me	是	获取当前用户
POST	/api/ai/analyze	否/建议是	分析自然语言
GET	/api/process-types	是	流程类型列表
GET	/api/process-types/{code}	是	流程类型详情
POST	/api/applications	是	创建申请
GET	/api/applications/my	是	我的申请分页
GET	/api/applications/{id}	是	申请详情
GET	/api/approvals/pending	是	待审批分页
POST	/api/approvals/{step_id}/approve	是	通过
POST	/api/approvals/{step_id}/reject	是	驳回
POST	/api/approvals/{step_id}/return	是	退回
POST	/api/upload	是	上传附件
GET	/api/dashboard/stats	是	仪表盘统计

2.3.3 关键接口详细定义

A. POST /api/ai/analyze

请求体由 AnalyzeRequest 校验，text 去除首尾空格后不能为空，长度为 1-2000。返回数据字段固定，可直接驱动动态表单和提示面板。

请求示例

```
{ "text": "我准备下周一请病假三天，工作交接给小王" }
```

响应结构示例（字段结构与源码一致，具体值取决于输入和模型）

```
{
  "code": 200,
  "message": "分析成功",
  "data": {
    "process_type": "leave",
    "confidence": 0.95,
    "title": "病假申请",
    "extracted_fields": { "leave_type": "病假", "days": 3, "handover_person": "小王" },
    "missing_fields": [ "start_date", "end_date", "reason" ],
    "required_materials": [ "病假证明" ],
    "approval_route": [ "manager", "hr" ],
    "risk_tips": [ "病假超过 2 天需上传证明" ],
    "summary": "..."
  }
}
```

B. POST /api/applications

请求示例

```
{
  "process_type": "reimbursement",
  "title": "差旅报销申请",
  "form_data": {
    "reimbursement_type": "差旅费",
    "amount": 3280.50,
  }
}
```

```
"expense_date": "2026-06-18",
"reason": "参加项目评审"
},
"ai_analysis": {"confidence": 0.94, "approval_route": ["manager","finance"]},
"attachment_ids": [12],
"remark": "原始票据已核对"
}
```

服务端校验 process_type 只能为三种支持类型，title 和 form_data 不得为空；创建时生成申请编号、规范化表单、校验材料、计算审批链并创建 ApprovalStep。

C. POST /api/approvals/{step_id}/{action}

请求体 (approve / reject / return 共用)

```
{"comment": "材料齐全，金额与事由一致，同意。"} 
```

ApprovalAction 要求 comment 长度 1-500 且去除空白后不能为空。服务层应检查当前用户是否为该步骤审批人、步骤是否 pending、申请是否仍可操作。通过非末节点时，下一 waiting 步骤转为 pending；末节点通过后申请完成；驳回/退回则记录终态或回退状态。

2.4 数据规格与状态规格

2.4.1 核心数据实体

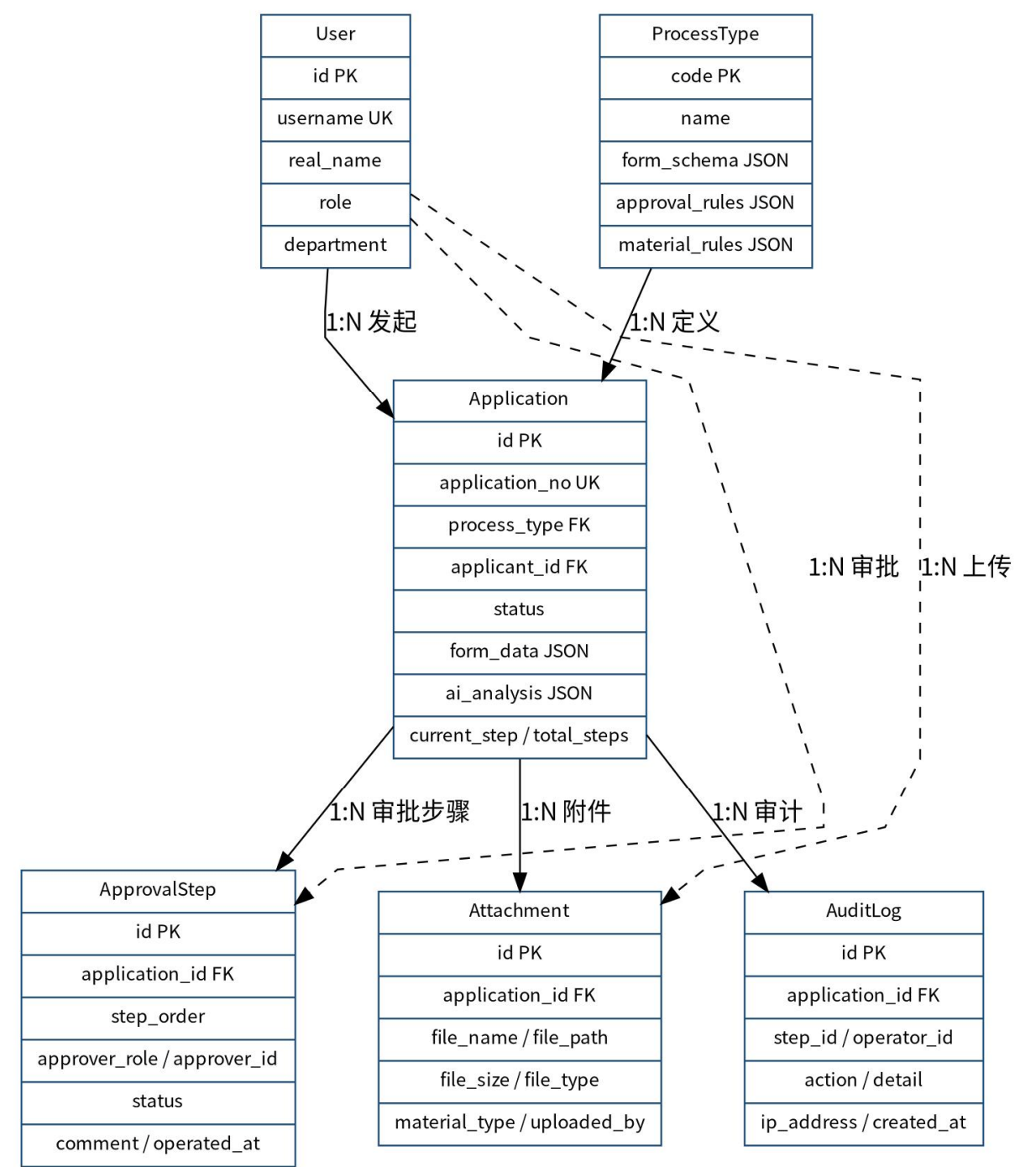


图 2-1 核心实体关系图（根据 SQLAlchemy 模型归纳）

实体	关键字段	职责
User	username、password_hash、real_name、role、department、is_active	身份、部门与角色权限
ProcessType	code、form_schema、approval_rules、material_rules	定义流程表单与规则
Application	application_no、process_type、applicant_id、status、form_data、ai_analysis、current_step	申请主记录

实体	关键字段	职责
ApprovalStep	application_id、step_order、approver_role、approver_id、status、comment	审批链实例与节点状态
Attachment	file_name、file_path、file_size、file_type、material_type、uploaded_by	附件元数据
AuditLog	application_id、step_id、operator_id、action、detail、ip_address	关键操作留痕

2.4.2 审批状态机

对象	状态	触发条件	下一状态/效果
ApprovalStep	waiting	步骤已创建但未轮到	前一步通过后→pending
ApprovalStep	pending	当前审批人可操作	approve→approved; reject→rejected; return→returned
Application	pending/processing	申请已提交且存在未完成步骤	节点推进，current_step 更新
Application	approved/completed	最后一步通过	流程完成，不再允许审批
Application	rejected	任一步驳回	流程终止，保留意见和日志
Application	returned	任一步退回	返回申请人修改或重新提交 (具体回转策略应明确)

2.5 需求—设计—实现追踪矩阵

需求	设计构件	实现位置	测试编号
FR-03 AI 分析	LangGraph 顺序图 + WorkflowState	app/ai/workflow.py, nodes.py, state.py	AT-01~AT-09
FR-05 动态表单	ProcessType.form_schema + 前端 SmartApply	models/process_type.py, views/SmartApply.vue	FT-05
FR-06 附件上传	UploadService + Attachment	routers/upload.py, services/upload_service.py	ST-07
FR-07 创建申请	ApplicationService + 事务	routers/application.py, services/application_service.py	FT-07
FR-10 待审批	角色/审批人过滤	routers/approval.py, approval_service.py	FT-10
FR-11 审批操作	ApprovalAction + 状态校验	schemas/approval.py, approval_service.py	FT-11~13
FR-12 状态推进	ApprovalStep 状态机	approval_service.py	BT-01~06
NFR 可降级	LLMClient + Mock 回退	llm_client.py, nodes.py, mock_engine.py	RT-01~03
NFR 安全	JWT、Depends、前端拦截器	core/security.py, deps.py, src/api/index.js	ST-01~06

三、系统架构与设计

3.1 核心架构图

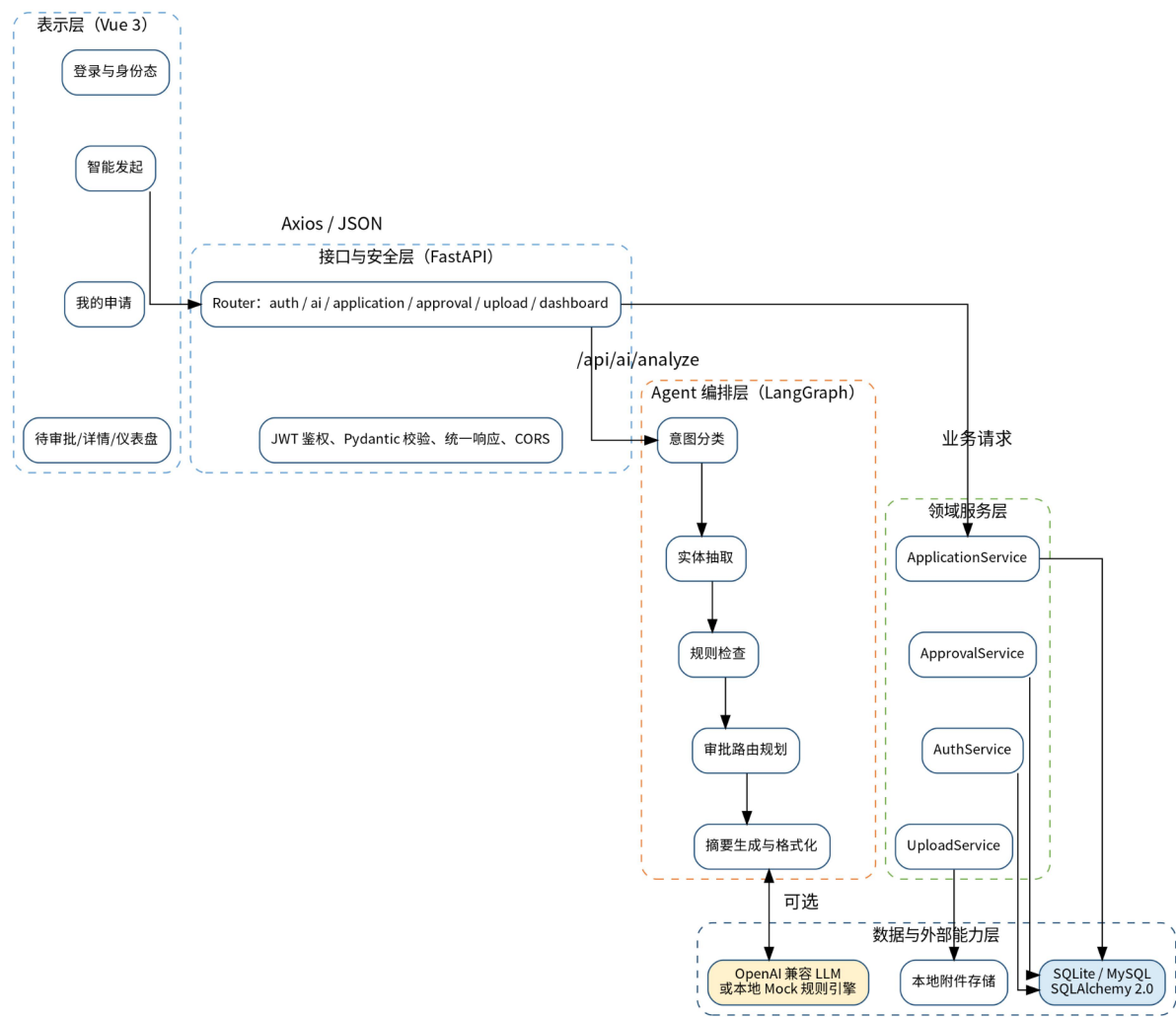


图 3-1 SmartFlow 分层架构与主要依赖

架构将“自然语言建议”与“确定性业务执行”分开：Agent 层负责理解、建议和结构化，领域服务层负责事务、权限、数据一致性和状态推进。这样的边界能够避免大模型直接修改数据库，同时保留 AI 对用户输入的适应能力。

3.2 Agent 交互流程

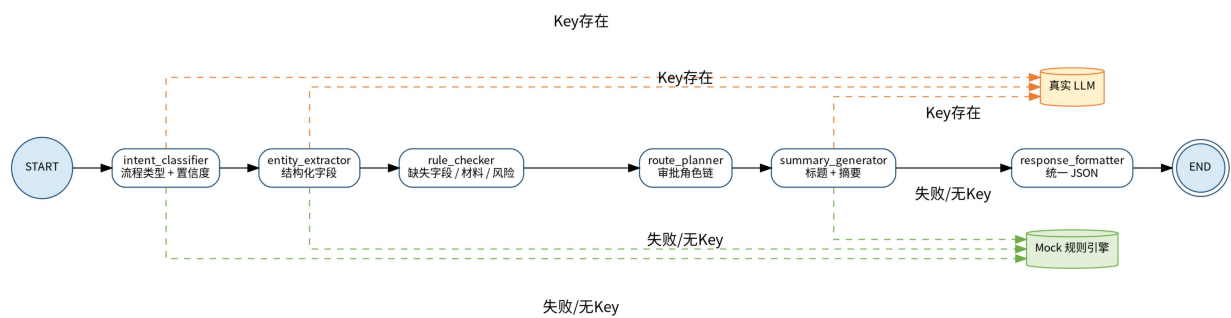


图 3-2 LangGraph Agent 节点与 LLM/Mock 双路径

节点	输入	输出	失败策略
intent_classifier	user_input	process_type, confidence	LLM 异常→关键词 Mock
entity_extractor	user_input, process_type	extracted_fields	LLM 异常→正则/规则抽取
rule_checker	process_type, extracted_fields	missing_fields, required_materials, risk_tips	纯函数规则，保持确定性
route_planner	process_type, extracted_fields	approval_route	金额/天数阈值规则
summary_generator	前述全部状态	title, summary	LLM 异常→模板生成
response_formatter	WorkflowState	final_response	缺省值归一化

架构判断：当前实现属于“有状态的 Agentic 工作流”：具备多节点编排、LLM 调用、规则工具与回退，但节点为固定顺序，尚未形成条件分支、循环反思、动态工具选择和人工中断恢复。报告第六章给出向更完整 Agent 架构演进的方案。

3.3 业务审批交互流程

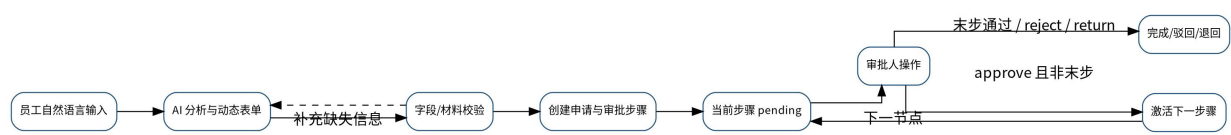


图 3-3 从自然语言到审批闭环的交互流程

提交阶段应坚持“AI 结果可编辑”：extracted_fields 只作为表单初值，用户补齐 missing_fields、核对金额/日期并上传 required_materials 后，才调用创建申请接口。审批阶段使用数据库中固化的 ApprovalStep，不在每次审批时重新让 LLM 决定下一节点，从而保证流程确定性。

3.4 数据流设计

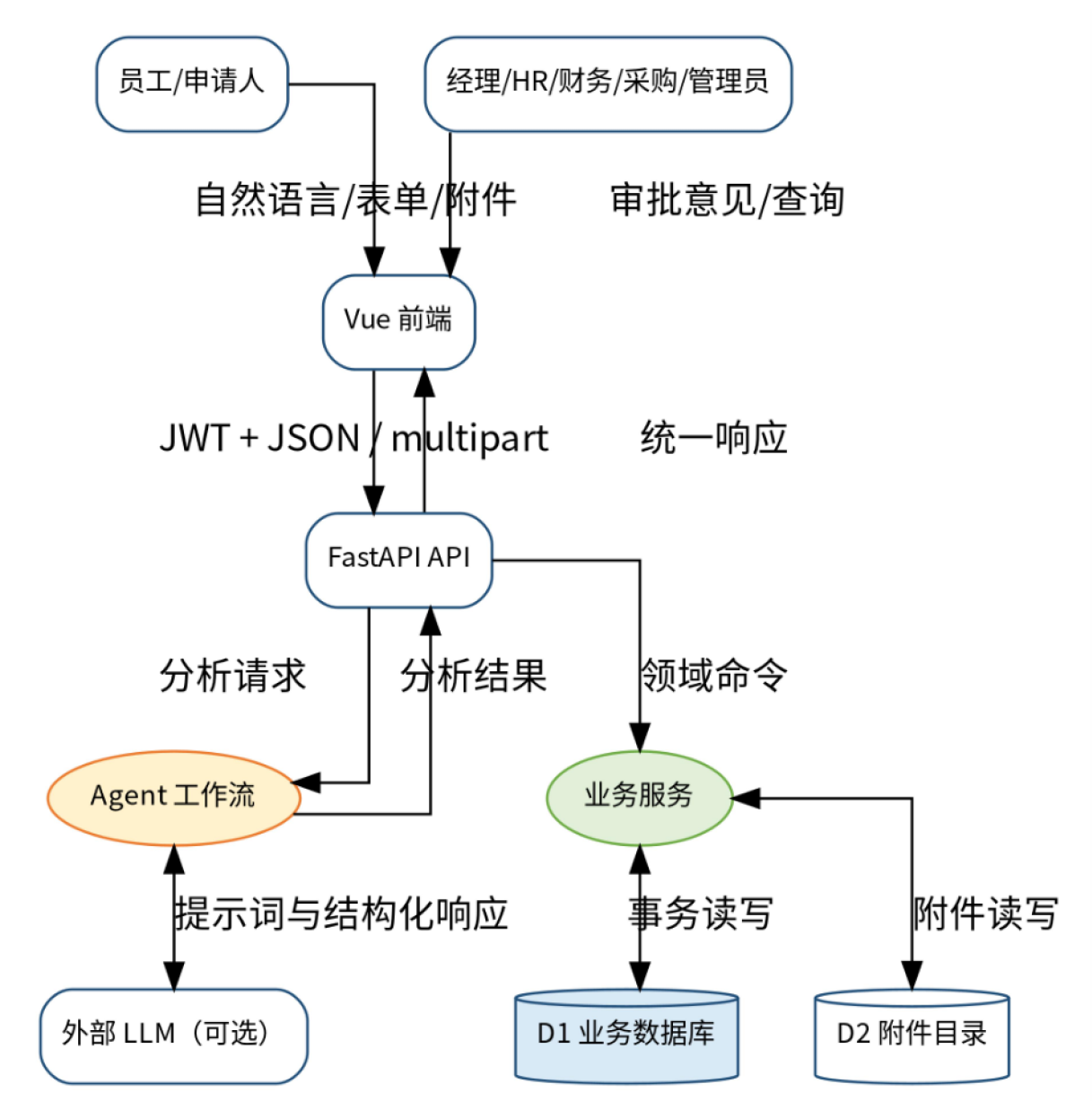


图 3-4 系统数据流图（DFD）

数据	生产者	消费者	存储/保护
自然语言文本	员工	Agent 节点、可选 LLM	默认不持久化原文或纳入 ai_analysis；生产环境需脱敏
结构化分析结果	Agent	前端、ApplicationService	随申请保存 ai_analysis，便于追溯
表单数据	员工确认	领域服务、审批详情	Application.form_data JSON
审批步骤	路由规则	审批列表、详情、状态机	ApprovalStep 表，有序且唯一
附件	员工	审批人	文件系统 + Attachment 元数据，需权限校验
审计事件	服务层	管理员、追责分析	AuditLog，记录操作者与 IP

3.5 部署与运行拓扑

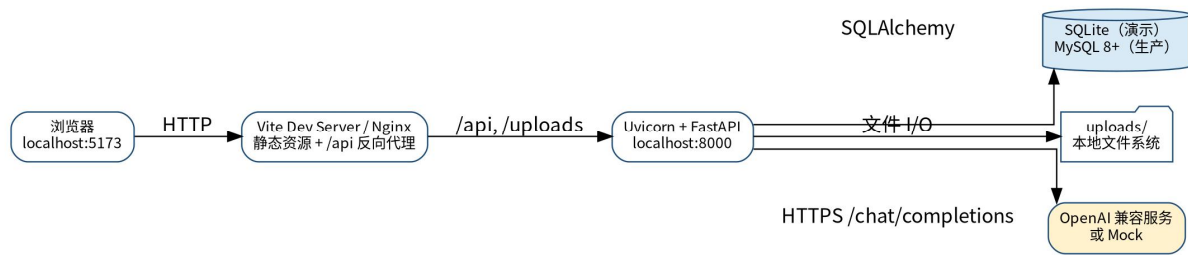


图 3-5 开发环境与可生产化部署拓扑

当前仓库采用 Vite 开发服务器 5173 端口和 FastAPI/Uvicorn 8000 端口，前端代理 /api 与 /uploads。演示环境使用 SQLite；生产环境可通过 DATABASE_URL 切换 MySQL。进一步生产化应使用 Nginx、Gunicorn/Uvicorn workers、对象存储、HTTPS、密钥管理和数据库备份。

四、关键实现与代码展示

4.1 后端应用装配

FastAPI 主程序注册 auth、ai、application、approval、process_type、upload、dashboard 等路由，配置 CORS 和静态上传目录。Router 只负责协议适配，复杂业务下沉到 services，形成较清晰的控制层—服务层—持久化层结构。

代码 4-1 FastAPI 路由装配（依据 main.py 结构整理）

```

app.include_router(auth.router, prefix="/api")
app.include_router(ai.router, prefix="/api")
app.include_router(application.router, prefix="/api")
app.include_router(approval.router, prefix="/api")
# CORS、StaticFiles 与异常处理在应用层统一配置

```

4.2 Agent 状态定义

WorkflowState 使用 TypedDict 定义节点之间共享的数据契约。每个节点返回局部更新，LangGraph 将结果合并到状态中。该设计使节点独立测试更容易，也避免直接传递非结构化上下文。

代码 4-2 Agent 共享状态（backend/app/ai/state.py）

```

class WorkflowState(TypedDict):
    user_input: str
    process_type: Optional[str]
    confidence: Optional[float]
    extracted_fields: Optional[dict[str, Any]]
    missing_fields: Optional[list[str]]
    required_materials: Optional[list[str]]
    risk_tips: Optional[list[str]]
    approval_route: Optional[list[str]]
    title: Optional[str]
    summary: Optional[str]
    final_response: Optional[dict[str, Any]]
    error: Optional[str]

```

4.3 Agent 核心循环/ workflow

源码中的 workflow 不是无限循环，而是一个可编译、可调用的有向图。核心执行链依次添加节点与边，入口传入完整初始状态，出口读取 `final_response`。其优势是执行路径可视化、节点可替换；当前不足是缺少条件边和重试节点。

代码 4-3 LangGraph 核心编排（依据 `workflow.py`）

```
workflow = StateGraph(WorkflowState)
workflow.add_node("intent_classifier", intent_classifier)
workflow.add_node("entity_extractor", entity_extractor)
workflow.add_node("rule_checker", rule_checker)
workflow.add_node("route_planner", route_planner)
workflow.add_node("summary_generator", summary_generator)
workflow.add_node("response_formatter", response_formatter)

workflow.add_edge(START, "intent_classifier")
workflow.add_edge("intent_classifier", "entity_extractor")
workflow.add_edge("entity_extractor", "rule_checker")
workflow.add_edge("rule_checker", "route_planner")
workflow.add_edge("route_planner", "summary_generator")
workflow.add_edge("summary_generator", "response_formatter")
workflow.add_edge("response_formatter", END)
compiled = workflow.compile()
```

4.4 LLM 调用与结构化 JSON 提取

LLMClient 通过 OpenAI 兼容的 `/chat/completions` 调用模型，`system message` 强制只返回 JSON，`temperature=0.2` 以降低波动，`httpx` 超时 30 秒。响应解析先尝试 fenced JSON，再尝试从文本中提取第一个对象。

代码 4-4 LLMClient 关键逻辑（`backend/app/ai/llm_client.py`）

```
payload = {
    "model": settings.llm_model,
    "messages": [
        {"role": "system", "content": "你是一个只返回 JSON 的企业审批助手。"},
        {"role": "user", "content": prompt},
    ],
    "temperature": 0.2,
}
response = client.post(f"{base_url}/chat/completions", json=payload, headers=headers)
content = response.json()["choices"][0]["message"]["content"]
return self._extract_json(content)
```

改进点：当前 `nodes.py` 使用 `except Exception: pass` 静默回退，保障了可用性，但降低了可观测性。建议记录 `node`、异常类型、耗时、模型、输入哈希和回退原因，且不能在日志中直接打印敏感原文。

4.5 意图、实体与摘要节点

代码 4-5 意图分类节点：真实 LLM 优先、Mock 回退

```
def intent_classifier(state):
    if client.enabled:
        try:
            result = client.chat_json(build_intent_prompt(state["user_input"]))
            if result.get("process_type") in {"leave", "reimbursement", "purchase"}:
                return {"process_type": result["process_type"],
                        "confidence": round(float(result.get("confidence", 0.8)), 2)}
        except Exception:
```

```
pass
process_type, confidence = classify_intent(state["user_input"])
return {"process_type": process_type, "confidence": confidence}
```

实体抽取节点同样遵循双模式；Prompt 为每类流程规定字段 schema，并要求“未知信息不编造”。摘要生成节点先构造模板 fallback，只有 LLM 同时返回 title 与 summary 时才覆盖，从而保证输出字段完整。

4.6 规则工具：材料、风险与审批路由

规则检查与路由规划使用确定性 helper 函数，而不是让 LLM 自由决定。这是系统最重要的工程控制：模型负责理解，规则负责合规。根据源码，主要阈值如下。

流程	业务量	审批角色链	材料/风险
请假	≤1 天	manager	基础请假信息
请假	1<天数≤3	manager → hr	病假超过 2 天需证明
请假	>3 天	manager → department_head → hr	长假需更多审批
报销	≤1000 元	manager	发票必需
报销	1000<金额≤5000	manager → finance	发票与事由核验
报销	>5000 元	manager → department_head → finance	高额报销风险提示
采购	≤2000 元	manager	基础采购信息
采购	2000<金额≤5000	manager → procurement	金额超过 3000 时需报价单
采购	>5000 元	manager → department_head → procurement → finance	高额采购多级审批

边界一致性：模板配置中若同时写 max=1000 与 min=1000，必须明确区间开闭。实际 helper 使用“≤1000”和“≤5000”的顺序判断，运行语义没有重叠；文档与配置应同步为 (1000,5000]。

4.7 申请创建与审批状态推进

ApplicationCreate 将 AI 分析结果与用户确认后的 form_data 分开保存。服务层应先规范化数据、验证必填字段和材料、生成申请编号，再创建有序审批步骤。审批动作的核心不是修改一个 status 字符串，而是同时更新步骤、申请 current_step、下一节点和审计日志。

代码 4-6 创建申请的输入边界

```
class ApplicationCreate(BaseModel):
    process_type: str
    title: str
    form_data: dict[str, Any]
    ai_analysis: dict[str, Any] | None = None
    attachment_ids: list[int] = Field(default_factory=list)
    remark: str | None = None

    @model_validator(mode="after")
    def validate_payload(self):
        if self.process_type not in {"leave", "reimbursement", "purchase"}:
            raise ValueError("不支持的流程类型")
        if not self.form_data:
            raise ValueError("表单数据不能为空")
        return self
```

ApprovalStep 对 application_id 与 step_order 设置唯一约束，防止同一申请生成重复序号。步骤初始通常为首步 pending、后续 waiting。approve 时激活下一节点；reject/return 记录审批意见和操作时间。

4.8 配置文件与环境切换

配置项	作用	建议值/注意事项
DATABASE_URL	数据库连接	开发 sqlite:///...；生产 mysql+pymysql://...
SECRET_KEY	JWT 签名密钥	生产必须使用高熵随机值并放入密钥管理系统
ACCESS_TOKEN_EXPIRE_MINUTES	令牌有效期	仓库示例 1440 分钟；敏感系统应缩短并支持刷新
LLM_API_KEY	是否启用真实模型	为空自动启用 Mock，不应提交真实 Key
LLM_BASE_URL	OpenAI 兼容端点	可接 OpenAI、DeepSeek 或企业网关
LLM_MODEL	模型名	按端点支持配置
MAX_UPLOAD_SIZE	上传体积上限	示例 10 MB；服务端和反向代理需一致
CORS_ORIGINS	允许前端来源	开发 localhost:5173；生产精确白名单

代码 4-7 本地启动命令

```
# 后端
cp .env.example .env
python init_db.py
python seed_data.py
uvicorn app.main:app --reload --port 8000

# 前端
npm install
npm run dev
```

4.9 AI IDE 使用过程证据



五、测试与评估

5.1 测试说明与证据边界

评估口径：以下“源码级结果”来自代码静态审查、输入约束和规则分支推演；未获得本机运行日志的项目标为“待运行确认”。报告不将静态推演伪装成真实压测或端到端运行结果。

测试分为四层：API/Schema 输入边界、Agent 节点行为、审批业务状态机、安全与异常恢复。最终提交时应在本机运行后补充 pytest 或接口测试截图、浏览器 Network 截图和完整 Demo 录屏。

5.2 功能测试用例

编号	场景	输入/前置	预期结果	当前判定
FT-01	正确账号登录	zhangsan/123456	200, 返回 access_token 与 user	源码路径明确：待运行确认
FT-02	错误密码登录	错误密码	401, 用户名或密码错误	源码级通过
FT-03	空 AI 输入	text=" "	Pydantic 校验失败	源码级通过
FT-04	超长 AI 输入	超过 2000 字	校验失败	源码级通过
FT-05	请假分析	病假 3 天并说明交接人	识别 leave, 生成字段/材料/路由	规则推演通过
FT-06	报销分析	差旅 3280 元	识别 reimbursement, 路由 manager→finance	规则推演通过
FT-07	采购分析	购买 10 台设备, 单价 800	识别 purchase, 总额 8000, 多级审批	规则推演通过
FT-08	创建空表单	form_data={}	拒绝, 表单数据不能为空	源码级通过
FT-09	不支持流程	process_type=travel	拒绝, 不支持的流程类型	源码级通过
FT-10	我的申请分页	page=1,page_size=10	仅返回当前用户数据	待运行/权限确认
FT-11	审批意见为空	comment=" "	校验失败	源码级通过
FT-12	通过中间步骤	当前 pending 节点 approve	当前 approved, 下一 waiting→pending	待运行确认
FT-13	通过最后步骤	末节点 approve	申请完成	待运行确认
FT-14	驳回申请	pending 节点 reject	申请 rejected, 记录意见	待运行确认
FT-15	退回申请	pending 节点 return	申请 returned/可修改策略生效	待运行确认
FT-16	无 LLM Key	LLM_API_KEY 为空	自动使用 Mock, 返回完整结构	源码级通过
FT-17	LLM 返回非 JSON	模拟文本响应	解析失败后回退 Mock	源码级通过
FT-18	上传非法扩展名	上传 .exe	拒绝	待运行确认
FT-19	401 处理	token 失效	前端清 token 并回登录页	README/前端设计确认
FT-20	切换 MySQL	修改 DATABASE_URL	初始化与种子数据成功	待环境确认

5.3 Agent 行为 Benchmark

Benchmark 重点不是只看“模型回答得像不像”，而是评估结构化正确性和业务可用性。建议指标：Intent Accuracy、Field Precision/Recall、Missing-field Recall、Route Exact Match、Material Rule Accuracy、JSON Valid Rate、Fallback Success Rate、平均延迟。

编号	自然语言样本	预期类型	预期路由	重点检查
----	--------	------	------	------

编号	自然语言样本	预期类型	预期路由	重点检查
AT-01	我明天家里有事，请一天事假	leave	manager	意图与天数
AT-02	下周住院，想请病假四天	leave	manager→department_head→hr	长假+病假证明
AT-03	昨天打车 86 元，报销客户拜访交通费	reimbursement	manager	金额/日期/原因
AT-04	参加会议差旅花了 3280 元，有发票	reimbursement	manager→finance	中额报销
AT-05	设备维修报销 8000 元	reimbursement	manager→department_head→finance	高额报销
AT-06	买 2 个键盘，每个 399 元	purchase	manager	数量×单价
AT-07	采购 5 台显示器，单价 900 元	purchase	manager→procurement	总额 4500+ 报价单
AT-08	采购服务器预算 25000 元	purchase	manager→department_head→procurement→finance	高额采购
AT-09	帮我办一下	不确定/默认低置信度	不应直接提交	歧义处理

指标	计算方式	建议门槛
Intent Accuracy	正确流程数 / 样本数	Mock 规则 ≥95%；真实 LLM ≥97%
Field F1	字段级精确率与召回率的调和平均	≥90%
Route Exact Match	审批角色序列完全一致比例	确定性规则应为 100%
Material Accuracy	必需材料集合完全一致比例	确定性规则应为 100%
JSON Valid Rate	可通过 Schema 校验的返回比例	真实 LLM ≥99%，含回退后 100%
Fallback Success	故障场景仍返回完整结构比例	100%
P95 Latency	95% 请求耗时	Mock <0.5s；LLM <8s（目标）

5.4 规则边界测试

编号	输入边界	预期审批链	预期材料
BT-01	请假 1 天	manager	按类型判断
BT-02	请假 3 天	manager→hr	病假且 >2 天需证明
BT-03	请假 4 天	manager→department_head→hr	长假规则
BT-04	报销 1000 元	manager	发票
BT-05	报销 1000.01 元	manager→finance	发票
BT-06	报销 5000 元	manager→finance	发票
BT-07	报销 5000.01 元	manager→department_head→finance	发票
BT-08	采购 2000 元	manager	基础材料
BT-09	采购 3000.01 元	manager→procurement	报价单
BT-10	采购 5000.01 元	manager→department_head→procurement→finance	报价单

5.5 安全与异常测试

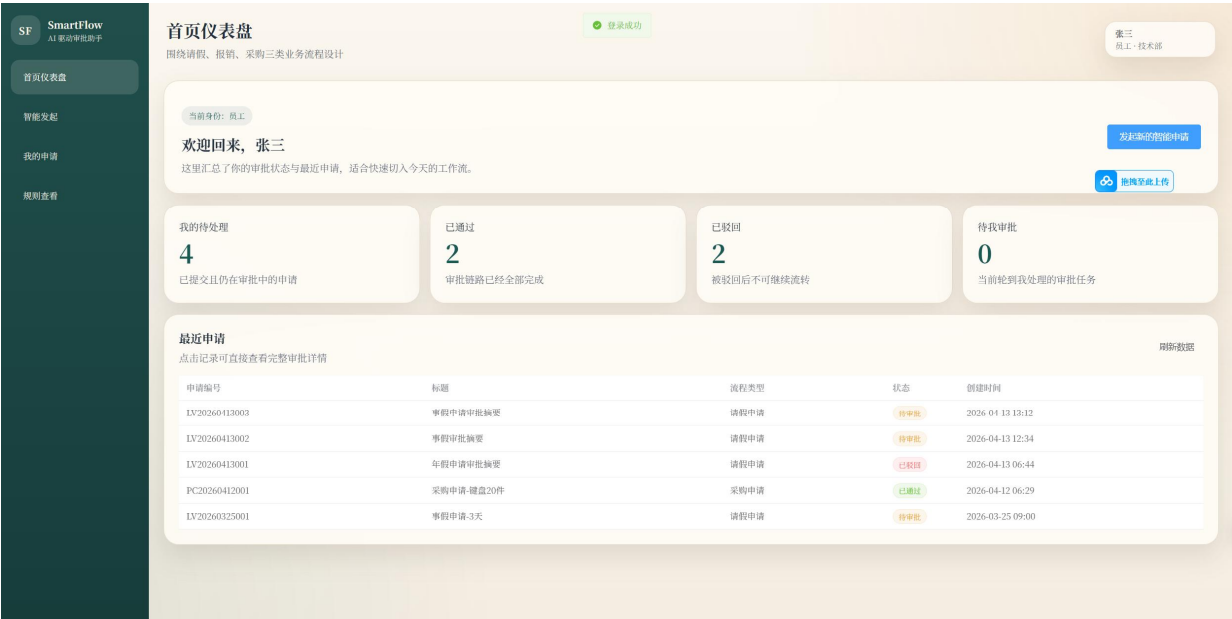
编号	测试	预期
ST-01	无 token 访问我的申请	401
ST-02	普通员工调用他人审批步骤	403/业务拒绝

编号	测试	预期
ST-03	重复审批同一 step	拒绝非 pending 状态
ST-04	修改 JWT role	签名验证失败
ST-05	上传路径穿越文件名	服务端生成 UUID 文件名，不使用原名作为路径
ST-06	恶意大文件	超过 MAX_UPLOAD_SIZE 拒绝
ST-07	LLM 超时	30 秒后异常并回退，不影响 API 结构
ST-08	数据库写入失败	事务回滚，不产生半条审批链
ST-09	跨域来源不在白名单	浏览器阻止/后端不返回允许头
ST-10	日志敏感信息	不得记录密码、token、LLM Key 和完整隐私文本

5.6 静态审查发现与缺陷清单

等级	发现	影响	建议
中	handover_person 在动态表单中可能为非必填，但 Mock get_missing_fields 将其列为必填	AI 提示与提交校验不一致	统一以 ProcessType.form_schema 为唯一必填源，并写一致性测试
中	病假证明规则需统一“≥2 天”还是“>2 天”	边界日数可能出现材料判断差异	规范文档明确开闭区间，模板与 helper 共享同一规则
中	LLM 异常被静默捕获	难以定位模型、网络或 JSON 问题	结构化日志 + fallback_reason + 指标
中	Agent 图为固定顺序，无条件分支	歧义输入仍会强制进入某流程	低置信度时转 clarification 节点
低	JWT 有效期示例 1440 分钟	共享设备风险增大	缩短 access token 并增加 refresh token/退出失效
低	本地文件系统存储附件	横向扩容与权限 URL 管理困难	迁移对象存储，使用短时签名链接
低	仓库暂未见系统化自动测试目录	回归风险	增加 pytest、Vitest、Playwright 与 CI

5.7 Demo 截图与录屏





六、系统升级与扩展

6.1 可扩展架构方向

方向	当前基础	下一步设计
流程扩展	三种 process_type + JSON form_schema	引入流程注册表/DSL，新流程以配置包和插件发布
Agent 编排	固定顺序节点	条件边、低置信度澄清、工具调用、人工中断与恢复
规则治理	Python helper 与模板配置	统一规则 DSL、版本号、生效日期、回放与审批
知识增强	Prompt 内硬编码字段	RAG 检索企业制度、费用标准、采购目录并返回引用
记忆与个性化	单次请求状态	用户偏好、常用成本中心、历史供应商，但需最小化隐私
模型治理	单一兼容端点	多模型路由、成本/延迟策略、评测集、提示词版本
系统集成	本地账号与数据库	SSO/LDAP、HR、财务、ERP、OA、消息通知和电子签章
运维	本地启动	容器化、CI/CD、监控、告警、备份和灰度发布

6.2 Agent 能力演进路线

阶段 1：可解释的结构化助手（当前）

自然语言转结构化字段，规则决定材料与审批链，用户确认后提交。重点是可靠、可降级和可演示。

阶段 2：对话澄清与工具调用

当 confidence 低或 missing_fields 较多时，不直接返回默认流程，而是进入 clarification 节点；Agent 可调用“查员工剩余假期”“查预算余额”“查供应商白名单”等受控工具。所有工具调用需参数 Schema、权限声明、超时和审计。

阶段 3：制度 RAG 与证据化决策

将企业制度切分、版本化并建立向量/关键词混合检索。Agent 返回每条材料要求和风险提示时附制度条款来源，规则引擎负责最终约束，LLM 只负责解释。

阶段 4：多 Agent 协作与流程优化

拆分为申请理解 Agent、合规审查 Agent、预算 Agent、审批路由 Agent 和总结 Agent；使用 supervisor 控制并行、冲突消解和人工审批。基于历史流程指标识别瓶颈，但不能让模型自行绕过合规节点。

6.3 目标 Agent 图

代码 6-1 下一阶段条件式 Agent 伪代码

```
START
-> classify
-> [confidence < threshold ? clarify_user : extract]
-> validate_schema
-> call_policy_search_tool
-> call_budget_or_leave_tool (按流程条件调用)
-> deterministic_rule_engine
-> human_review_interrupt
-> persist_application
-> END
```

控制原则：工具必须白名单化；写操作必须在人机确认后执行；制度与金额规则必须由确定性引擎裁决；模型输出要通过 Pydantic/JSON Schema；每次决策保留 trace 与证据。

6.4 工程化升级

模块	升级项	收益
后端	异步 httpx、连接池、Alembic、Redis 缓存、Celery/RQ	提升吞吐、迁移与长任务处理能力
前端	表单草稿、断点续传、可访问性、错误边界、E2E 测试	提升用户体验和质量
数据	MySQL 主从/备份、对象存储、字段级加密、数据保留策略	满足可靠性与合规
安全	RBAC/ABAC、短期 token、CSRF/限流、文件病毒扫描、密钥轮换	降低越权和攻击面
可观测	OpenTelemetry、结构化日志、Prometheus 指标、LLM 成本统计	定位性能与模型回退
质量	pytest、contract test、Vitest、Playwright、GitHub Actions	自动回归和持续交付

6.5 迭代计划

周期	里程碑	交付物	验收
第 1-2 周	质量基线	自动测试、修复规则一致性、日志	核心规则边界 100% 通过
第 3-4 周	对话澄清	条件图、clarification UI、会话状态	歧义样本不再强制误分类
第 5-6 周	制度 RAG	制度库、引用展示、版本管理	材料/风险提示可追溯

周期	里程碑	交付物	验收
第 7-8 周	企业集成	SSO、消息通知、对象存储	端到端集成演示
第 9-12 周	生产化	容器、监控、备份、压测与安全评审	达到 SLA 与上线检查表

七、课程总结

7.1 个人收获

本项目最大的收获是认识到企业级 AI 应用不能停留在“调用一次模型并展示文本”。真正可用的系统需要明确输入输出契约、规则边界、权限、事务、降级、审计和测试。SmartFlow 中，LLM 负责将自然语言映射为候选结构，Pydantic 和流程规则负责约束，领域服务负责最终业务执行，这种分工比把全部判断交给模型更可靠。

通过阅读和组织项目代码，进一步理解了前后端分离、Router-Service-Model 分层、JWT 身份态、动态表单、审批状态机、文件上传和数据库关系建模。LangGraph 的价值也不只是“多写几个函数”，而是把 AI 任务拆成可观察、可替换、可评估的节点，为后续条件分支和工具调用提供结构。

7.2 工程思维的转变

过去的倾向	本项目后的认识
先写页面和接口，再补文档	先用 Product/Architecture/API Spec 定义边界，再实现和追踪
只关注正常路径	必须设计无 Key、LLM 超时、JSON 异常、越权、重复审批等失败路径
认为 AI 输出就是结果	AI 输出只是建议，必须经 Schema、规则与人工确认
认为功能完成等于质量完成	还需要自动测试、可观测、审计、安全和部署能力
把规则散落在条件语句中	规则应版本化、可测试、可解释，并与流程模板保持一致

7.3 对课程的建议

- 增加一次“需求—架构—API—测试追踪矩阵”评审，使同学不只展示界面。
- 要求 Agent 项目明确区分模型推理、确定性规则和写操作权限，避免把聊天功能等同于 Agent。
- 增加运行证据规范，例如测试日志、AI IDE 过程、Git 提交、Demo 录屏和问题复盘。
- 提供企业系统中的安全、审计、可观测和数据合规案例，使项目更接近真实软件工程。

7.4 项目结论

SmartFlow 已形成一个结构完整的企业审批演示系统：自然语言输入通过 LangGraph 节点转为可解释的结构化分析，用户确认后创建申请，确定性规则生成审批链，角色用户完成多步审批，附件、时间线和审计日志提供过程证据。项目在教学演示、架构清晰度和离线可运行方面具有优势。下一阶段应优先解决规则一致性、自动测试、异常可观测和歧义澄清，再扩展制度 RAG、受控工具和企业集成。

附录 A：部署与演示操作手册

A.1 环境要求

软件	要求
Python	3.10+
Node.js	18+
npm	9+
数据库	SQLite（默认）或 MySQL 8.0+
浏览器	Chrome/Edge 新版

A.2 启动步骤

1. 进入 backend，安装 requirements.txt，复制 .env.example 为 .env。
2. 执行 python init_db.py 与 python seed_data.py。
3. 执行 uvicorn app.main:app --reload --port 8000。
4. 进入 frontend，执行 npm install 与 npm run dev。
5. 访问 <http://localhost:5173>；Swagger 为 <http://localhost:8000/docs>。

A.3 演示账号

用户名	密码	角色	部门/用途
zhangsan	123456	employee	员工发起申请
lisi	123456	manager	经理审批
wangwu	123456	finance	财务审批
zhaoliu	123456	hr	HR 审批
sunqi	123456	procurement	采购审批
admin	123456	admin	管理员/兜底

安全提示：演示账号和密码仅适用于本地课程环境。公开部署前必须删除或修改种子账号，启用强密码和 HTTPS。

附录 B：提交前检查清单

检查项	完成标准	完成
封面	学号、姓名、专业已替换；指导教师与日期核对	☐
目录	Ctrl+A、F9 更新；PDF 导航窗格可用	☐
AI IDE	至少 2-3 张本人真实使用截图，含提示词和代码变更	☐
代码展示	路径与代码保持和最终仓库一致	☐
测试	补充真实运行截图、测试日志与实际通过率	☐
Demo	加入 4 张关键页面截图或录屏链接/二维码	☐
仓库	确认链接可访问，敏感 Key、数据库与上传隐私未提交	☐
PDF	检查图片清晰、表格不跨页错位、字体不乱码	☐
真实性	静态推演与实测结果明确区分，不伪造截图或数据	☐
文件名	按课程要求命名为 docs/CS599_大作业报告.pdf	☐

附录 C：参考资料与源码定位

SmartFlow GitHub 仓库： <https://github.com/ffyyssw/smartflow>（项目 README、前后端源码与配置）

FastAPI Documentation： <https://fastapi.tiangolo.com/>（后端接口与依赖注入框架）

LangGraph Documentation： <https://langchain-ai.github.io/langgraph/>（Agent 状态图与节点编排）

Vue.js Guide： <https://vuejs.org/guide/introduction.html>（前端框架）

SQLAlchemy 2.0 Documentation： <https://docs.sqlalchemy.org/en/20/>（ORM 与事务）

Pydantic Documentation： <https://docs.pydantic.dev/>（输入输出模型和校验）

C.1 关键源码定位

主题	文件
应用入口	backend/app/main.py
Agent 工作流	backend/app/ai/workflow.py
Agent 节点	backend/app/ai/nodes.py
共享状态	backend/app/ai/state.py
模型客户端	backend/app/ai/llm_client.py
Mock 引擎	backend/app/ai/mock_engine.py
Prompt	backend/app/ai/prompts.py
规则工具	backend/app/utils/helpers.py
认证接口	backend/app/routers/auth.py
AI 接口	backend/app/routers/ai.py
申请接口	backend/app/routers/application.py
审批接口	backend/app/routers/approval.py
申请模型	backend/app/models/application.py
审批步骤模型	backend/app/models/approval_step.py
前端智能发起	frontend/src/views/SmartApply.vue
前端 API	frontend/src/api/
环境配置	backend/.env.example, frontend/vite.config.js

— 报告正文结束 —